LAB EXERCISE: 3

Graph Retrieval-Augmented Generation (RAG) — Natural-Language Question Answering over the Neo4j Movies Knowledge Graph

A modular Streamlit web application that links each question to movies and people in the Neo4j Movies graph, retrieves the relevant subgraph with parameterised Cypher queries, and grounds an LLM-generated answer in the retrieved facts.

Aim / Objective

This project implements a simple but complete Retrieval-Augmented Generation (RAG) system that uses the Neo4j Movies dataset as its knowledge source. Instead of letting a large language model (LLM) answer movie questions from its own parametric memory — where it can silently hallucinate cast lists, directors or release years — the system first retrieves the relevant facts from a graph database and then asks the LLM to answer only from those facts.

The pipeline is organised as reusable modules: a graph store (graph_store.py / graph_db.py) that executes Cypher against Neo4j, a GraphRetriever that turns a free-text question into linked entities, intents and a ranked list of graph facts, and a generator that builds a grounded prompt and calls an LLM (Google Gemini / OpenAI-compatible / local engine). The Streamlit front end (app.py) shows the answer next to the evidence it was built from: the retrieved facts, the drawn subgraph, the execution trace, and the Cypher that was executed.

The system was evaluated on a hand-written set of 30 questions covering eleven answerable question types, three deliberately hard types and one out-of-graph question, so that both its strengths and its blind spots are reported.

Expected Outcomes Addressed

- **1. Use a graph database as the knowledge source.** The official Neo4j Movies script (movies.cypher) is loaded into Neo4j, giving 171 nodes and 253 relationships. All retrieval goes through named, parameterised Cypher templates — user text is never concatenated into a query.
- **2. Retrieve relevant movie information for a user query.** Movie titles and person names are linked to graph nodes (exact phrase, unique surname, or fuzzy match); keyword rules detect which relationship types are being asked about and any release-year constraint; the 1-hop neighbourhood of every linked node (plus a shared-movie path for two people) is fetched and ranked.
- **3. Build a grounded context for the LLM.** Retrieved facts are serialised as numbered (subject) -[RELATION]-> (object) [detail] triples and placed under a system prompt that forbids outside knowledge and prescribes a fixed refusal sentence when the answer is missing.
- **4. Generate natural-language answers with an LLM.** LLMGenerator posts the prompt to the Google Gemini (gemini-3.8-flash) endpoint at low temperature. If offline, a deterministic template generator verbalises the same facts so the app always answers.
- **5. Present and evaluate the system.** A Streamlit UI exposes the answer with its evidence, and evaluation test suites score entity linking, context recall and answer accuracy across test questions.

Tools & Technologies Used

Category	Tool / Library	Purpose in this pipeline
Language / runtime	Python 3	Core implementation language for the whole system.
Knowledge source	Neo4j 5 (graph DB) + Movies dataset	Stores Movie and Person nodes and their ACTED_IN / DIRECTED / WROTE / PRODUCED / REVIEWED / FOLLOWS relationships.
Graph access	neo4j Python driver, Cypher	Parameterised Cypher templates for movie profiles, person profiles, shared-movie paths and year ranges.
Entity linking	Regex & RapidFuzz	Fuzzy string matching of misspelt titles and names (ratio ≥ 88).
Fallback retrieval	Hybrid Entity Search	Neighbourhood subgraph traversal when no exact entity is linked.
Generation	Google Gemini (gemini-3.8-flash) via google.genai	Writes the final answer from the grounded prompt at low temperature.
Web framework	Streamlit	Sidebar configuration, question input, answer card, tabs, CSV download.
Graphs	Vis.js Force Network	Draws the retrieved subgraph as an interactive node–edge chart.
Tabular data	pandas	Facts table, CSV export and evaluation results.

Dataset / Corpus Description

The knowledge source is the Neo4j Movies example graph — the dataset Neo4j ships for its :play movies tutorial and publishes as movies.cypher in the neo4j-graph-examples/movies repository. It is a small, curated property graph rather than a text corpus, which makes it well suited to RAG: every fact is explicit, typed and individually retrievable.

Element	Count	Properties / notes
Movie nodes	38	title, released (1975–2012), tagline (missing for 1 movie)
Person nodes	133	name, born (missing for 5 people)
ACTED_IN	172	roles (list of character names)
DIRECTED	44	—
PRODUCED	15	—
WROTE	10	—
REVIEWED	9	summary, rating (0–100)
FOLLOWS	3	Person → Person (not used for retrieval in this version)
Total	**171 nodes / 253 rels**	**Verified after loading.**

No cleaning is applied: the script is loaded exactly as published, so the system inherits the dataset's own values.

System Architecture / Pipeline Design

Stage	What happens
1. Input	The user types a question (or picks an example) and clicks “Ask”; the sidebar selects the graph back end and displays Gemini AI status.
2. Entity linking	link_entities() matches the graph's titles and names against the normalised question, longest first, so “The Matrix Reloaded” is consumed before “The Matrix”.
3. Intent & constraints	detect_intents() maps keywords to relationship types (directed → DIRECTED, played/role → ACTED_IN, …) and parses year constraints into bounds.
4. Graph retrieval	For each linked movie, movie_profile; for each person, person_profile; for two people, shared_movies; for a year range alone, movies_by_year.
5. Fallback retrieval	If nothing was linked, hybrid subgraph traversal returns the closest connected entities.
6. Ranking & context	Each fact is scored (+2 if its relation matches an intent, +2 if it touches a second linked entity), de-duplicated and cut to top k triples.
7. Prompt & generation	build_prompt() adds system rules; Google Gemini (gemini-3.8-flash) answers at low temperature, or semantic template generator verbalises facts.
8. Streamlit UI	Answer card, execution metrics, and tabs for facts table, interactive force-directed subgraph, LLM prompt and Cypher used.

Backend Implementation

6.1 Approach / Algorithm

Retrieval is graph-native and entity-first. Because every answer in the Movies graph hangs off a named movie or person, the retriever's first job is to find those names in the question; the graph then supplies a precise, complete neighbourhood for each one in a single Cypher round-trip. This avoids the main weakness of chunk-based RAG on structured data — a cast list split across chunks, or a director mentioned in a chunk that never scores highly — because the unit of retrieval is the node's whole 1-hop subgraph.

Generation is constrained, not creative: the system prompt restricts the model to the numbered facts, asks for concise factual sentences with release years, and fixes the refusal sentence. Temperature 0 keeps answers repeatable. Every component is configurable from environment variables (NEO4J_URI, GEMINI_API_KEY).

Sample Input & Output

Input (question)	Pipeline stage	Output produced
“Who played Neo in The Matrix?”	Linking → retrieval → generation	Linked movie: The Matrix; intent ROLE; 5 facts. Answer: “In The Matrix (1999), the cast includes Keanu Reeves (as Neo).”
“When was Keanu Reeves born?”	Fuzzy entity linking	RapidFuzz links “keanu reeves” to Keanu Reeves; the BORN fact gives “Keanu Reeves was born in 1964.”
“Which movies did Tom Hanks act in after 2000?”	Year-constrained retrieval	“after 2000” → range 2001+; person_profile is filtered to 4 films. Answer: “Tom Hanks acted in The Polar Express (2004), The Da Vinci Code (2006), Charlie Wilson's War (2007) and Cloud Atlas (2012).”
“What rating did Jessica Thompson give Unforgiven?”	Edge-property retrieval	The REVIEWED edge's rating and summary are returned: “Unforgiven (1992) was reviewed by Jessica Thompson (rating 85: 'Dark, but compelling').”
“Which movie has the tagline Freedom! Forever!?”	Property fallback	Lexical match over taglines ranks V for Vendetta first: “The closest match in the graph is V for Vendetta (2005), tagline: 'Freedom! Forever!'.”
“Who directed Titanic?”	Out-of-graph refusal	Nothing linked in movie graph, so context is empty and fixed refusal is returned: “I could not find that in the movie graph.”

Results & Quantitative Evaluation

Setting	Value
Graph back end	Neo4j AuraDB Cloud / Built-in High-Fidelity Movies Graph (171 nodes, 253 rels)
Generator	Google Gemini (gemini-3.8-flash) via google.genai / Semantic Synthesizer
Context size / threshold	max_facts = 25, Temperature = 0.0 - 0.2

Metric	Value	Definition
Entity-linking accuracy	100% (30/30)	Linked movies + people exactly equal expected set.
Context recall	90% (27/30)	Every gold answer string appears in retrieved context.
Answer accuracy	90% (27/30)	Every gold string appears in final answer (or refusal).
Mean facts per context	8.6 (range 0–23)	Triples sent to the generator.
Retrieval latency	1.33 ms mean	Linking + retrieval + ranking on graph.

Conclusion

The project delivers a working Graph RAG system over the Neo4j Movies dataset: questions are linked to graph entities, the relevant subgraph is retrieved with parameterised Cypher, and an LLM is asked to answer strictly from the retrieved, numbered facts, with every step visible in the Streamlit interface. On a 30-question test set, entity linking was correct for every question, context recall and answer accuracy were both 90%, and all entity-centred question types — including typos, surname-only names, year filters and two-person questions — were answered correctly, while an out-of-graph question was correctly refused.

The evaluation also shows where a simple design stops working: a lexical fallback that can retrieve the wrong movie without warning, no support for aggregation or character-name questions, and noisy contexts for multi-entity questions. Each is localised to a single component and can be improved without changing the overall retrieve-then-generate architecture.

Project Repository (GitHub Link)

Field	Value
GitHub Repository URL	https://github.com/Manishankar-614/Neo4j_Movies.git
Repository Visibility	Public
Branch / Commit Used for Submission	main

References

- Neo4j Movies example graph — https://github.com/neo4j-graph-examples/movies
- Neo4j Python driver manual — https://neo4j.com/docs/python-manual/current/
- Neo4j Cypher manual — https://neo4j.com/docs/cypher-manual/current/
- P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS 2020.
- Google Gemini API documentation — https://ai.google.dev/gemini-api/docs
- Streamlit documentation — https://docs.streamlit.io
- Vis.js Network documentation — https://visjs.github.io/vis-network/docs/network/

ADVANCE NLP (AML23702) — Lab Exercise Evaluation Sheet

Field	Value
Student Name	**Manishankar**
USN	**1GA23AI025**
Lab Exercise No.	3
Experiment Title	Graph Retrieval-Augmented Generation (RAG) — Natural-Language Question Answering over the Neo4j Movies Knowledge Graph

Marks Awarded

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Problem Understanding & Scope	5	
2	Pipeline Design & Methodology	10	
3	Implementation Correctness	10	
4	Use of NLP Tools / Models	5	
5	Output Quality / Task Performance	10	
6	Analysis & Interpretation	5	
7	Documentation & Deliverables	2.5	
8	Presentation & Reproducibility	2.5	
Total	**Total Marks**	**50**	

Overall Comments

Date of Submission: _____ Signature of Faculty: _____